

这份PPT（Disjoint Set & Class）详细介绍了不相交集（Disjoint Set）数据结构及其应用，主要围绕等价关系、等价类、以及实现不相交集操作（Union/Find）的各种策略和优化。

以下是内容的详细总结，采用Markdown格式：

Lec.8 PPT内容总结：不相交集类（Disjoint Set Class）

1. 等价关系（Equivalence Relations）

- **定义：**关系 R 定义在集合 S 上，对于 S 中的每对元素 a, b ， aRb 或者为真，或者为假。
- **等价关系必须满足的三个性质：**
 - **自反性 (Reflexive)：**对于所有 $a \in S$ ，有 aRa 。
 - **对称性 (Symmetric)：**若 aRb ，则 bRa 。
 - **传递性 (Transitive)：**若 aRb 且 bRc ，则 aRc 。
- **示例：**关系“=”是等价关系。

2. 等价类（Equivalence Classes）

- **定义：**元素 $a \in S$ 的等价类是 S 的一个子集，包含所有与 a 相关的元素。
- **性质：**
 - S 中不同的等价类是**不相交的** (Disjoint)。
 - S 中的每个成员恰好出现在一个等价类中。

3. 等价问题与动态等价问题（Equivalence Problem & Dynamic Equivalence Problem）

- **等价问题：**给定一个等价关系 R ，判断一对元素 $a, b \in S$ 是否满足 aRb 。
- **动态等价问题：**这是一个动态（在线）问题，因为集合会在操作过程中发生变化，要求 Find 操作必须能应对这种变化。
 - **策略：**从每个元素作为一个单元素集开始（这些单元素集是不相交的）。
 - **操作：**
 - **Find：**查找给定元素的等价类（集合）。
 - **Union：**联合两个集合。

4. 不相交集的联合/查找（Disjoint Union/Find, D-U/F）

- **数据结构：**一组两两不相交的集合（Disjoint Sets）。
- **集合命名：**每个集合都有一个唯一的名称，通常是其成员之一。
- **操作：**
 - **Find(x)：**返回包含元素 x 的集合的名称（即根节点）。
 - **Union(x, y)：**将名为 x 和 y 的两个集合合并为一个集合。
- **实现困境：**两种基本策略：一个策略确保 Find 操作在最坏情况下为 $O(1)$ ，另一个确保 Union 操作在最坏情况下为 $O(1)$ 。

5. 向上树（Up-Tree）的实现

- **表示：**使用**向上树（Up-Tree）**表示每个集合，每个元素有一个指向父节点的链接。
- **数组 $s[i]$ ：** $s[i]$ 存储元素 i 的父节点。
- **简单 Union 实现：**Union(root1, root2) 假设 root1 和 root2 是根节点，将 root2 指向 root1。
- **简单 Find 实现：**Find(x) 沿着父链接向上直到根节点并返回根节点。

- **不良情况 (Bad Case)**: 简单的 Union 实现可能导致树退化成链表, 使得 Find(1) 在最坏情况下需要 N 步操作 (即 $O(N)$)。

6. 改进策略: 优化 Union/Find

6.1. 加权联合 (Weighted Union / Union-by-Size)

- **策略**: 在 Union 操作中, 总是将**节点较少**的树的根指向**节点较多**的树的根。
- **空间优化**: 为了节省空间, 根节点 $s[i]$ 不存储 -1 , 而是存储其集合大小的负值。
- **性能分析**: 使用加权联合, 高度为 h 的向上树至少有 2^h 个节点。

6.2. 按高度联合 (Union-by-Height)

- **策略**: 类似于加权联合, 但根据树的高度进行合并。根节点存储树高度的负值。
 - 如果两树高度不同, 将矮树的根指向高树的根。
 - 如果两树高度相同, 任意选择一个作为新根, 并将新根的高度减 1。

6.3. 路径压缩 (Path Compression)

- **策略**: 在执行 Find 操作时, 将搜索路径上的所有节点直接指向根节点。
- **Find 实现**: 通过递归实现路径压缩。

```
int DisjSets::find( int x ) {
    if( s[ x ] < 0 )
        return x; // 根节点
    else
        return s[ x ] = find( s[ x ] ); // 递归并压缩路径
}
...
```

6.4. 综合优化: 加权联合 + 路径压缩

- **最坏情况时间复杂度**: Union 为 $O(1)$, Find 为 $O(\log N)$ 。
- **均摊复杂度 (Amortized Complexity)**: 对于 N 个元素上的 $m \geq N$ 次操作, 时间复杂度为 $O(m \log^* N)$ 。
 - $\log^* N$ 是迭代对数, 增长非常缓慢 (例如 $\log^* 2^{65536} = 5$)。
 - 这意味着平均到每次操作, 时间复杂度**基本上是常数时间**。

7. 应用: 构建随机迷宫 (Building a Random Maze)

- **目标迷宫的特性**:
 - 边界不被删除。
 - 每个单元格都可以到达其他每个单元格 (连通性)。
 - 没有环路 (Cycle, 即隐藏的树结构)。
- **D-U/F 基本算法**:
 - 初始化**: 集合 S 中每个单元格 (cell) 作为独立的集合; E 为所有墙壁 (边) 的集合; $Maze$ (要保留的墙) 为空。
 - 循环**: 当 S 中集合数量多于一个时:
 - 随机选取一条边 (x, y) 并从 E 中移除。
 - 查找 $u := Find(x)$; $v := Find(y)$ 。
 - **如果** $u \neq v$: 调用 $Union(u, v)$ (敲掉墙壁, 将两个集合连接起来)。
 - **如果** $u = v$: 将边 (x, y) 加入 $Maze$ (保留墙壁, 因为 x 和 y 之间已经存在路径, 再敲会形成环路)。
 - 结果**: 所有剩余的边和 $Maze$ 中的边一起构成了迷宫 (即墙壁)。